

ЛАБОРАТОРНАЯ РАБОТА №3: «Настройка защиты и анализ трафика»

Легенда: ты — сетевой администратор и специалист по безопасности. Твоя задача — настроить firewall на сервере, проанализировать сетевой трафик и обнаружить подозрительную активность.

Задача 1. Анализ сетевых интерфейсов и маршрутизации

Шаг 1.1. Посмотри все сетевые интерфейсы в системе и их IP-адреса. Определи, какой интерфейс активен и какой IP-адрес использует твоя система.

используем команду `ip addr show` для вывода всех сетевых интерфейсов

```
(kali@kali)-[~]
$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
    inet 172.20.10.5/28 brd 172.20.10.15 scope global dynamic noprefixroute eth0
        valid_lft 3561sec preferred_lft 3561sec
    inet6 fe80::1766:6d25:d5dc:c877/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

видим здесь lo, это у нас loopback адрес, это у нас петля, куда возвращается весь трафик. видим стандартный IP 127.0.0.1

второй интерфейс это eth0, это уже для взаимодействия с внешней средой как я поняла. тут используется IP 172.20.10.15

Шаг 1.2. Посмотри таблицу маршрутизации. Определи, какой IP-адрес у шлюза по умолчанию (роутера).

для просмотра таблицы маршрутизации используем команду `ip route`

```
(kali@kali)-[~]
$ ip route
default via 172.20.10.1 dev eth0 proto dhcp src 172.20.10.5 metric 100
172.20.10.0/28 dev eth0 proto kernel scope link src 172.20.10.5 metric 100
```

первая строчка это наверное маршрут по умолчанию, судя по слову default, ну или шлюз по умолчанию судя по единице на конце IP адреса 172.20.10.1, далее написано dhcp src 172.20.10.5, судя по dhcp, адрес 172.20.10.5 был выдан интерфейсу eth0 с метрикой 100. получается шлюз по умолчанию имеет адрес 172.20.10.1

во второй строчке я так понимаю написан уже маршрут по умолчанию 172.20.10.0 с маской /28 то есть 255.255.255.240. то есть получается весь незнакомый трафик с интерфейса eth0 уходит на шлюз по умолчанию, указанный выше по данному маршруту.

Шаг 1.3. Посмотри ARP-таблицу. Сколько устройств в локальной сети ты видишь? Есть ли что-то подозрительное?

для открытия arp таблицы использовали команду `ip neigh show`

```
(kali@kali)-[~]
$ ip neigh show
172.20.10.1 dev eth0 lladdr d2:6b:78:7d:f1:64 REACHABLE
fe80::d06b:78ff:fe7d:f164 dev eth0 lladdr d2:6b:78:7d:f1:64 router STALE
```

это таблица соответствия IP-MAC. видим, что роутеру (шлюзу по умолчанию) соответствует MAC адрес d2:6b:78:7d:f1:64. тут в первой строчке у нас написано соответствие роутера с адресом 172.20.10.1 данному MAC. а во второй строчке судя по тому, что MAC тот же самый, это тот же самый IP адрес, только версии 6 (IPv6). получается вывод: устройство обнаружено одно, это роутер наш, через который выходим в интернет. ничего подозрительного не обнаружено.

Задача 2. Настройка firewall (UFW)

Шаг 2.1. Проверь текущий статус firewall. Включён ли он? Какие правила уже настроены?

проверим текущий статус firewall при помощи команды `sudo iptables -L -n -v`

```
(kali@kali)-[~]
$ sudo iptables -L -n -v
[sudo] password for kali:
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination
```

видим три цепочки: INPUT (входящие пакеты. пример: подключение по SSH к нашему серверу, пинг нашего адреса), FORWARD (транзитные пакеты. пример: компьютер из нашей локальной сети запрашивает сайт в интернете через наш сервер-шлюз), OUTPUT (исходящие пакеты. пример: выполнение команды `apt update`, обновление системы или скачивание файла из интернета прямо с сервера). в Linux iptables работает непосредственно на уровне ядра и включен всегда по умолчанию. но он не выполняет никакой фильтрации, работает в режиме полного пропуска. на данный момент правила не настроены, так как списки под каждой из цепочек пусты. для всех трех цепочек

указано policy ACCEPT. это значит, что любые пакеты автоматически разрешены. значения 0 packets 0 bytes показывают, что с момента запуска или сброса счетчиков через эти цепочки еще не проходили пакеты.

Шаг 2.2. Настрой базовую защиту:

- Установи политику по умолчанию: блокировать все входящие соединения
- Разрешить все исходящие соединения
- Разрешить SSH (порт 22)
- Разрешить HTTP (порт 80) и HTTPS (порт 443)
- Включи firewall

политика по умолчанию: `sudo iptables -P INPUT DROP`

разрешить исходящие: `sudo iptables -P OUTPUT ACCEPT`

разрешить SSH: `sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT`

разрешить HTTP, HTTPS: `sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT` и `sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT`

я почитала в интернете что при отключении входного трафика надо обязательно разрешить локальный трафик lo, так как это может отключить важные системные службы

`sudo iptables -A INPUT -i lo -j ACCEPT`

```

(kali㉿kali)-[~]
$ sudo iptables -P INPUT DROP

(kali㉿kali)-[~]
$ sudo iptables -P OUTPUT ACCEPT

(kali㉿kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport=22 -j ACCEPT

(kali㉿kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT

(kali㉿kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

(kali㉿kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

(kali㉿kali)-[~]
$ sudo iptables -A INPUT -i lo -j ACCEPT

```

комментарий: в первых двух командах мы писали DROP или ACCEPT без флага -j, так как мы меняем всю цепочку а не просто прописываем правила.

```

(kali㉿kali)-[~]
$ sudo iptables -L -n -v
Chain INPUT (policy DROP 388 packets, 26534 bytes)
 pkts bytes target    prot opt in     out     source    destination
  0    0 ACCEPT    tcp  --  *      *       0.0.0.0/0  0.0.0.0/0    tcp dpt:22
  0    0 ACCEPT    tcp  --  *      *       0.0.0.0/0  0.0.0.0/0    tcp dpt:80
  0    0 ACCEPT    tcp  --  *      *       0.0.0.0/0  0.0.0.0/0    tcp dpt:443
  0    0 ACCEPT    tcp  --  *      *       0.0.0.0/0  0.0.0.0/0    tcp dpt:22
  0    0 ACCEPT    all  --  lo     *       0.0.0.0/0  0.0.0.0/0

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination

Chain OUTPUT (policy ACCEPT 353 packets, 22379 bytes)
 pkts bytes target    prot opt in     out     source    destination

```

увидели применение этих правил. все работает

Шаг 2.3. Добавь правило для ограничения SSH (защита от брутфорса).

используем команду `sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --update --seconds 60 --hitcount 4 --name SSH_BF -j DROP`

если IP подключается 4-й раз в течение минуты, его пакеты будут молча сбрасывать в DROP

Как работает эта команда

- `sudo iptables -A INPUT`: Добавляет правило в цепочку INPUT (для всех входящих пакетов).
- `-p tcp --dport 22`: Правило применяется только к протоколу **TCP** и целевому порту **22** (стандартный порт SSH).
- `-m conntrack --ctstate NEW`: Отслеживает только **новые** попытки соединения (уже установленные сессии не разорвутся).
- `-m recent`: Включает модуль recent для динамического отслеживания IP-адресов.
- `--update`: Проверяет, есть ли IP-адрес в списке. Если есть, обновляет время его последней активности.
- `--seconds 60`: Задает временное окно в **60 секунд**.
- `--hitcount 4`: Устанавливает лимит в **4 попытки** соединения за указанное время.
- `--name SSH_BF`: Сохраняет список атакующих IP-адресов в файл с именем **SSH_BF** (его можно посмотреть в `/proc/net/xt_recent/SSH_BF`).
- `-j DROP`: **Сбрасывает** (блокирует) пакеты, если лимит превышен. Клиент не получит никакого ответа.

а если прям правильно, то должно быть так

1. Сначала блокируем тех, кто превысил лимит (4 подключения за 60 секунд)

```
sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --update --seconds 60 --hitcount 4 --name SSH_BF -j DROP
```

2. Если лимит не превышен, добавляем IP в список и разрешаем вход

```
sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --set --name SSH_BF -j ACCEPT
```

Разбор команды №1 (Блокировка нарушителей)

```
sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --update --seconds 60 --hitcount 4 --name SSH_BF -j DROP
```

Эта команда срабатывает **только** для тех IP-адресов, которые уже есть в списке SSH_BF и нарушили лимит.

- **-m conntrack --ctstate NEW**: Модуль отслеживания соединений. Фильтрует только пакеты, которые пытаются открыть **новое** SSH-соединение. Попытки ввода пароля внутри уже открытой сессии здесь не учитываются.
- **-m recent --update**: Модуль recent ищет IP-адрес отправителя в своей базе данных SSH_BF. Если адрес найден, модуль обновляет время его последнего пакета.
- **--seconds 60 --hitcount 4**: Это условие проверки. Модуль смотрит: «Было ли от этого IP 4 или более подключений за последние 60 секунд?».

- **--name SSH_BF**: Имя специального файла в памяти ядра (/proc/net/xt_recent/SSH_BF), где хранятся IP-адреса и метки времени их подключений.
- **-j DROP**: Действие. Если IP-адрес найден в списке и условие (4 подключения за минуту) выполнено, пакет **немедленно сбрасывается**. Сервер делает вид, что его не существует. Проверка на этом прекращается.

Разбор команды №2 (Регистрация и пропуск)

```
sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --set --name SSH_BF -j ACCEPT
```

До этой команды пакет дойдет только в двух случаях: IP-адреса еще **нет в списке SSH_BF**, либо он там есть, но **не превысил лимит** (например, это только второе подключение).

- **-m recent --set**: Главное отличие от первой команды. Этот флаг берет IP-адрес отправителя и **заносят его в список SSH_BF** (или обновляет существующую запись, если он там уже был). При этом в файл записывается временная метка текущего подключения.
- **-j ACCEPT**: Действие. Пакет успешно **пропускается** к SSH-демону. Пользователь видит приглашение ввести пароль.

iptables-recent (или модуль recent) — это встроенный инструмент межсетевого экрана Linux, который позволяет **динамически создавать списки IP-адресов** и проверять их в режиме реального времени.

Если обычные правила iptables работают статично (например, просто блокируют конкретный IP), то модуль recent добавляет **логику и память**. Он запоминает, кто, когда и сколько раз стучался на ваш сервер.

Основные параметры (флаги) модуля

Чтобы управлять этим модулем, используются четыре ключевых флага:

1. **--set (Запомнить)**

Берет IP-адрес текущего пакета и добавляет его в список. Если этот IP уже есть в списке, модуль просто обновляет время его последнего появления.

2. **--update (Проверить и обновить)**

Проверяет, есть ли IP-адрес в списке. Если есть — обновляет время его последней активности. Если адреса в списке нет, правило просто игнорируется (не срабатывает).

3. **--rcheck (Просто проверить)**

Похож на --update, но только проверяет наличие IP в списке, **не обновляя** время его

последнего пакета. Это важно, если вы не хотите, чтобы частые атаки продлевали время блокировки.

4. **--remove (Удалить)**

Ищет IP-адрес в списке и, если находит, полностью удаляет его оттуда. Полезно для создания систем «стука в порты» (Port Knocking), когда правильный пакет снимает блокировку.

Шаг 2.4. Включи логирование заблокированных пакетов.

ПРАВИЛО 1: Смотрит на нарушителя, ПИШЕТ ЛОГ и передает пакет дальше

```
sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --update --seconds 60 --hitcount 4 --name SSH_BF -j LOG --log-prefix "SSH-BRUTEFORCE: "
```

ПРАВИЛО 2: Смотрит на того же нарушителя и НАГЛУХО СБРАСЫВАЕТ пакет

```
sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --update --seconds 60 --hitcount 4 --name SSH_BF -j DROP
```

Шаг 2.5. Проверь, что все правила применены корректно.

в общем если собрать все воедино получаем такой порядок

1. Если лимит превышен (4 подключения за 60 сек), обновляем метку времени нарушителя

Но здесь мы пакет пока НЕ сбрасываем, а просто проверяем условие

```
sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --update --seconds 60 --hitcount 4 --name SSH_BF -j LOG --log-prefix "SSH-BRUTEFORCE-DROP: " --log-level 4
```

2. Сразу за логом отправляем этот же пакет в DROP (блокируем)

```
sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --update --seconds 60 --hitcount 4 --name SSH_BF -j DROP
```

3. Если лимит не превышен — добавляем IP в список и разрешаем вход

```
sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --set --name SSH_BF -j ACCEPT
```

```
(kali@kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --update --seconds 60 --hitcount 4 --name SSH_BF -j LOG --log-prefix "SSH-BRUTEFORCE-DROP: " --log-level 4
[sudo] password for kali:

(kali@kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --update --seconds 60 --hitcount 4 --name SSH_BF -j DROP

(kali@kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --set --name SSH_BF -j ACCEPT
```

Задача 3. Анализ трафика с помощью tcpdump

Шаг 3.1. Захвати 20 пакетов на активном интерфейсе и проанализируй их. Какие протоколы ты видишь?

пользуемся командой `sudo tcpdump -i eth0 -c 20`

```
(kali@kali)-[~]
$ sudo tcpdump -i eth0 -c 20
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
09:04:57.976719 IP 172.20.10.5.51222 > 149.154.167.99.https: Flags [P.], seq 2042448361:2042448501, ack 3188311036, win 323, options [nop,nop,TS val 1685006655 ecr 529664211], length 140
09:04:57.981808 IP 172.20.10.5.42120 > 172.20.10.1.domain: 25176* PTR? 99.167.154.149.in-addr.arpa. (45)
09:04:57.938118 IP 172.20.10.1.domain > 172.20.10.5.42120: 25176 NXDomain 0/1/0 (138)
09:04:57.938465 IP 172.20.10.5.36132 > 172.20.10.1.domain: 17215* PTR? 5.10.20.172.in-addr.arpa. (42)
09:04:57.964931 IP 149.154.167.99.https > 172.20.10.5.51222: Flags [P.], seq 1:121, ack 140, win 127, options [nop,nop,TS val 529666218 ecr 1685006655], length 120
09:04:57.964976 IP 172.20.10.5.51222 > 149.154.167.99.https: Flags [.], ack 121, win 323, options [nop,nop,TS val 1685006743 ecr 529666210], length 0
09:04:57.976819 IP 172.20.10.1.domain > 172.20.10.5.36132: 17215 NXDomain 0/1/0 (101)
09:04:58.000484 IP 172.20.10.5.39225 > 172.20.10.1.domain: 55133* PTR? 1.10.20.172.in-addr.arpa. (42)
09:04:58.036258 IP 172.20.10.1.domain > 172.20.10.5.39225: 55133 NXDomain 0/1/0 (101)
09:04:59.099374 IP 172.20.10.5.51222 > 149.154.167.99.https: Flags [P.], seq 140:330, ack 121, win 323, options [nop,nop,TS val 1685006670 ecr 529666210], length 190
09:04:59.988480 IP 149.154.167.99.https > 172.20.10.5.51222: Flags [P.], seq 121:241, ack 330, win 127, options [nop,nop,TS val 529668230 ecr 168500678], length 120
09:04:59.988526 IP 172.20.10.5.51222 > 149.154.167.99.https: Flags [.], ack 241, win 323, options [nop,nop,TS val 1685008767 ecr 529668230], length 0
09:05:01.893336 IP 172.20.10.5.51222 > 149.154.167.99.https: Flags [P.], seq 330:536, ack 241, win 323, options [nop,nop,TS val 1685010672 ecr 529668230], length 206
09:05:01.971042 IP 149.154.167.99.https > 172.20.10.5.51222: Flags [P.], seq 241:361, ack 536, win 127, options [nop,nop,TS val 529670211 ecr 1685010672], length 120
09:05:01.971084 IP 172.20.10.5.51222 > 149.154.167.99.https: Flags [.], ack 361, win 323, options [nop,nop,TS val 1685010750 ecr 529670211], length 0
09:05:02.860952 IP 149.154.167.99.https > 172.20.10.5.51222: Flags [P.], seq 361:497, ack 536, win 127, options [nop,nop,TS val 529671097 ecr 1685010750], length 136
09:05:02.861034 IP 172.20.10.5.51222 > 149.154.167.99.https: Flags [.], ack 497, win 323, options [nop,nop,TS val 1685011639 ecr 529671097], length 0
09:05:02.894915 IP 172.20.10.5.51222 > 149.154.167.99.https: Flags [P.], seq 536:676, ack 497, win 323, options [nop,nop,TS val 1685011673 ecr 529671097], length 140
09:05:02.995306 IP 149.154.167.99.https > 172.20.10.5.51222: Flags [P.], seq 676:850, ack 497, win 323, options [nop,nop,TS val 529671243 ecr 1685011673], length 0
09:05:03.901864 IP 172.20.10.5.51222 > 149.154.167.99.https: Flags [P.], seq 676:850, ack 497, win 323, options [nop,nop,TS val 1685012680 ecr 529671243], length 174
20 packets captured
22 packets received by filter
0 packets dropped by kernel
```

я вижу только протокол IP во всех 20 пакетах. это какие-то пакеты по обмену данными в интернете, трафик переходит с одного IP на другой.

Шаг 3.2. Захвати только DNS-трафик (порт 53). Какие домены запрашивает твоя система?

```
(kali@kali)-[~]
$ sudo tcpdump -i eth0 port 53
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
09:14:27.404146 IP 172.20.10.5.50705 > 172.20.10.1.domain: 44825+ A? kws2-1.web.telegram.org. (41)
09:14:27.404190 IP 172.20.10.5.50705 > 172.20.10.1.domain: 51040+ AAAA? kws2-1.web.telegram.org. (41)
09:14:27.409137 IP 172.20.10.5.46471 > 172.20.10.1.domain: 32488+ HTTPS? web.telegram.org. (34)
09:14:27.412168 IP 172.20.10.5.56793 > 172.20.10.1.domain: 13584+ A? web.telegram.org. (34)
09:14:27.412206 IP 172.20.10.5.56793 > 172.20.10.1.domain: 62476+ AAAA? web.telegram.org. (34)
09:14:27.416797 IP 172.20.10.5.39816 > 172.20.10.1.domain: 35232+ A? web.telegram.org. (34)
09:14:27.475782 IP 172.20.10.1.domain > 172.20.10.5.50705: 51040 1/0/0 AAAA 2001:67c:4e8:f004::9 (69)
09:14:27.476878 IP 172.20.10.1.domain > 172.20.10.5.50705: 44825 1/0/0 A 149.154.167.99 (57)
09:14:27.478663 IP 172.20.10.1.domain > 172.20.10.5.56793: 62476 1/0/0 AAAA 2001:67c:4e8:f004::9 (62)
09:14:27.478664 IP 172.20.10.1.domain > 172.20.10.5.56793: 13584 1/0/0 A 149.154.167.99 (50)
09:14:27.478664 IP 172.20.10.1.domain > 172.20.10.5.39816: 35232 1/0/0 A 149.154.167.99 (50)
09:14:27.480052 IP 172.20.10.5.47567 > 172.20.10.1.domain: 37936+ A? kws2-1.web.telegram.org. (41)
09:14:27.480568 IP 172.20.10.1.domain > 172.20.10.5.46471: 32488 0/1/0 (113)
09:14:27.482582 IP 172.20.10.5.39267 > 172.20.10.1.domain: 18574+ PTR? 1.10.20.172.in-addr.arpa. (42)
09:14:27.503227 IP 172.20.10.1.domain > 172.20.10.5.47567: 37936 1/0/0 A 149.154.167.99 (57)
09:14:27.503227 IP 172.20.10.1.domain > 172.20.10.5.39267: 18574 NXDomain* 0/1/0 (101)
09:14:27.503492 IP 172.20.10.5.43901 > 172.20.10.1.domain: 15006+ PTR? 5.10.20.172.in-addr.arpa. (42)
09:14:27.521589 IP 172.20.10.1.domain > 172.20.10.5.43901: 15006 NXDomain* 0/1/0 (101)
09:14:27.974378 IP 172.20.10.5.36077 > 172.20.10.1.domain: 43294+ A? kws2-1.web.telegram.org. (41)
09:14:27.974386 IP 172.20.10.5.36077 > 172.20.10.1.domain: 42780+ AAAA? kws2-1.web.telegram.org. (41)
09:14:27.987152 IP 172.20.10.1.domain > 172.20.10.5.36077: 43294 1/0/0 A 149.154.167.99 (57)
09:14:27.987153 IP 172.20.10.1.domain > 172.20.10.5.36077: 42780 1/0/0 AAAA 2001:67c:4e8:f004::9 (69)
09:15:39.865255 IP 172.20.10.5.35532 > 172.20.10.1.domain: 61842+ A? push.services.mozilla.com. (43)
09:15:39.865312 IP 172.20.10.5.35532 > 172.20.10.1.domain: 18064+ AAAA? push.services.mozilla.com. (43)
09:15:39.900043 IP 172.20.10.1.domain > 172.20.10.5.35532: 61842 1/0/0 A 34.107.243.93 (59)
09:15:39.900043 IP 172.20.10.1.domain > 172.20.10.5.35532: 18064 0/1/0 (124)
09:15:40.720740 IP 172.20.10.5.58527 > 172.20.10.1.domain: 75+ A? firefox.settings.services.mozilla.com. (55)
09:15:40.761893 IP 172.20.10.1.domain > 172.20.10.5.58527: 75 2/0/0 CNAME mozilla.map.fastly.net., A 146.75.117.91 (107)
```

судя по выводу, тут одни домены telegram и mozilla с firefox, ну то есть браузер и мессенджер, которые у меня как раз открыты.

Шаг 3.3. Захвати HTTP-трафик (порт 80) и покажи содержимое пакетов в ASCII-формате. Что ты можешь увидеть?

```
(kali㉿kali)-[~]  
$ sudo tcpdump -i eth0 port 80 -A  
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode  
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes  
^[B^[B^[B^[B^[B^[B^[B^[B^[B^Z  
zsh: suspended sudo tcpdump -i eth0 port 80 -A
```

тут как-то все странно в выводе, но может я что-то не так ввела

Шаг 3.4. Захвати трафик и сохрани его в файл `capture.pcap`. Открой этот файл в Wireshark (если установлен) или проанализируй через `tcpdump -r capture.pcap`.

Задача 4. Обнаружение подозрительной активности

Шаг 4.1. Проверь, не слушает ли кто-нибудь в системе классические "хакерские" порты (4444, 1337, 31337, 8888, 9999).

```
(kali@kali)-[~]
$ sudo ss -tulpn
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
tcp	LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	users(("sshd",pid=697,fd=6))
tcp	LISTEN	0	128	:::22	:::*	users(("sshd",pid=697,fd=7))

проверили при помощи команды `ss -tulnp`, видим, что есть только порт 22, только он слушается.

Шаг 4.2. Захвати трафик на подозрительных портах (если они есть).

—

Шаг 4.3. Проверь, нет ли в системе установленных соединений с внешними IP-адресами на нестандартных портах.

ну нет нестандартных портов

Шаг 4.4. Просканируй свою систему с помощью netcat: найди все открытые порты на localhost (127.0.0.1).

```
(kali㉿kali)-[~]  
$ nc -zv 127.0.0.1  
no port[s] to connect to
```

команда показала, что подключенных портов нет

Задача 5. Создание и анализ reverse shell (в безопасной среде)

Шаг 5.1. Открой два терминала. В первом терминале запусти netcat в режиме слушания на порту 4444.

```
(kali@kali)-[~]  
$ nc -l -p 4444
```

подключились и стали слушать порт 4444 на первом терминале

Шаг 5.2. Во втором терминале подключись к localhost на порт 4444 с помощью netcat.

```
(kali@kali)-[~]  
$ nc 127.0.0.1 4444
```

подключились во втором терминале

Шаг 5.3. Попробуй отправить сообщение из одного терминала в другой.

```
(kali@kali)-[~]  
$ nc -l -p 4444  
echo > "Hello!!!!!"  
echo > "Im hacker"
```

```
(kali@kali)-[~]  
$ nc 127.0.0.1 4444  
echo > "Hello!!!!!"  
echo > "Im hacker"
```

отправляли сообщения с одного терминала на другой

Шаг 5.4. Захвати этот трафик с помощью tcpdump и проанализируй его.

```
kali@kali: ~  
Session Actions Edit View Help  
[kali@kali]~  
$ nc 127.0.0.1 4444  
ndicc  
decwawcaw  
sudo tcpdump -i lo port 4444  
yauoyh  
nsijcpnpe  
veivb1bebhve  
hwc1bqbchwbecjhqbec  
hbqJNWQJCKLCW  
[  
kali@kali: ~  
Session Actions Edit View Help  
[kali@kali]~  
$ sudo tcpdump -i lo port 4444  
[sudo] password for kali:  
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode  
listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes  
10:45:52.622629 IP localhost.44074 > localhost.4444: Flags [P.], seq 1746007436:1746007446, ack 3145914061, win 256, options [nop,nop,TS val 1230106326 ecr 1230055204], length 10  
10:45:52.622655 IP localhost.4444 > localhost.44074: Flags [.], ack 10, win 256, options [nop,nop,TS val 1230106326 ecr 1230106326], length 0  
10:45:54.416862 IP localhost.44074 > localhost.4444: Flags [P.], seq 10:23, ack 1, win 256, options [nop,nop,TS val 1230108120 ecr 1230106326], length 13  
10:45:54.416876 IP localhost.4444 > localhost.44074: Flags [.], ack 23, win 256, options [nop,nop,TS val 1230108120 ecr 1230108120], length 0  
10:45:56.140602 IP localhost.44074 > localhost.4444: Flags [P.], seq 23:43, ack 1, win 256, options [nop,nop,TS val 1230109844 ecr 1230108120], length 20  
10:45:56.140622 IP localhost.4444 > localhost.44074: Flags [.], ack 43, win 256, options [nop,nop,TS val 1230109844 ecr 1230109844], length 0  
10:45:57.544769 IP localhost.44074 > localhost.4444: Flags [P.], seq 43:58, ack 1, win 256, options [nop,nop,TS val 1230111248 ecr 1230109844], length 15  
10:45:57.544788 IP localhost.4444 > localhost.44074: Flags [.], ack 58, win 256, options [nop,nop,TS val 1230111248 ecr 1230111248], length 0  
[
```

я в третьем терминале начала перехват трафика. а во втором ввела несколько сообщений. итого в третьем высветилось как раз таки наши эти сообщения. показано, что идет пересылка что-то типа чата, то есть отправка с localhost.44074 > localhost.4444. а потом ack localhost.4444 > localhost.44074, то есть я так понимаю подтверждение доставки следующей строчкой.

Шаг 5.5. Заверши оба соединения.

завершено.